

K L Deemed to be University

Department of Computer Science and Engineering | 2026–2027, Odd Semester
Database Systems Engineering and Distributed Backend Development (25CS1302E)

SKILLING SESSION 4 – TRANSACTIONS AND STORED LOGIC

Name	Chakrika Panakala	Roll No.	2520090042
Course	25CS1302E	CO Mapping	CO1
Session	Skilling 4	Topic	Transactions and Stored Logic

Aim

To demonstrate the ACID properties with BEGIN/COMMIT/ROLLBACK/SAVEPOINT, observe MVCC row versions, compare isolation levels and the concurrency anomalies (dirty read, non-repeatable read, phantom read), use advisory locks, and implement PL/pgSQL functions, stored procedures, an audit trigger, views and materialised views for Task Hub.

1. Database Preparation and Budget Hours

```
\c taskhub_db

ALTER TABLE projects
ADD COLUMN budget_hours NUMERIC(6,1) NOT NULL DEFAULT 100
CHECK (budget_hours >= 0);

UPDATE projects SET budget_hours = 120 WHERE project_id = 1;
UPDATE projects SET budget_hours = 80 WHERE project_id = 2;

SELECT project_id, name, budget_hours
FROM projects ORDER BY project_id;
```

Output

project_id	name	budget_hours
1	Task Hub Backend	120.0
2	Task Hub Frontend	80.0
3	Mobile App	100.0

2. ACID – COMMIT

Atomicity is demonstrated by transferring 20 budget hours from project 1 to project 2 inside one transaction. Both updates are committed together.

```
BEGIN;
UPDATE projects SET budget_hours = budget_hours - 20 WHERE project_id = 1;
UPDATE projects SET budget_hours = budget_hours + 20 WHERE project_id = 2;
COMMIT;

SELECT project_id, budget_hours
FROM projects WHERE project_id IN (1,2)
ORDER BY project_id;

1 | 100.0
2 | 100.0
```

3. ACID – ROLLBACK

A CHECK constraint prevents a negative budget. When the second update violates the constraint, the transaction is rolled back and the earlier update is also undone.

```
BEGIN;
UPDATE projects SET budget_hours = budget_hours + 500 WHERE project_id = 2;
UPDATE projects SET budget_hours = budget_hours - 500 WHERE project_id = 1;
ROLLBACK;

ERROR: new row for relation "projects" violates check constraint
"projects_budget_hours_check"

After ROLLBACK:
1 | 100.0
2 | 100.0
```

4. SAVEPOINT

A savepoint is used to undo only the second insert while retaining the first insert.

```
BEGIN;
INSERT INTO tasks (project_id, assignee_id, title, status)
VALUES (1, 3, 'Write unit tests', 'TODO');

SAVEPOINT after_first_task;
```

```
INSERT INTO tasks (project_id, assignee_id, title, status)
VALUES (1, 3, 'Wrong task', 'TODO');

ROLLBACK TO SAVEPOINT after_first_task;
COMMIT;

SELECT title FROM tasks
WHERE title IN ('Write unit tests', 'Wrong task');

title
-----
Write unit tests
```

5. MVCC – Row Versions

PostgreSQL uses MVCC. The xmin value identifies the transaction that created a row version, while xmax records the transaction that deleted or locked it. Updating a row creates a new row version.

```
SELECT xmin, xmax, project_id, budget_hours
FROM projects WHERE project_id = 1;
```

```
UPDATE projects
SET budget_hours = budget_hours
WHERE project_id = 1;
```

```
SELECT xmin, xmax, project_id, budget_hours
FROM projects WHERE project_id = 1;
```

Before UPDATE:

xmin	xmax	project_id	budget_hours
1034	1036	1	100.0

After UPDATE:

xmin	xmax	project_id	budget_hours
1038	0	1	100.0

6. Isolation Levels

```
SHOW default_transaction_isolation;
-- read committed
```

```
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SHOW transaction_isolation;
-- repeatable read
COMMIT;
```

7. Advisory Locks

```
SELECT pg_try_advisory_lock(42) AS lock_acquired;
SELECT locktype, objid, mode, granted
FROM pg_locks WHERE locktype = 'advisory';
```

```
SELECT pg_advisory_unlock(42) AS lock_released;
```

```
lock_acquired
-----
t
```

locktype	objid	mode	granted
advisory	42	ExclusiveLock	t

```
lock_released
-----
t
```

8. PL/pgSQL Function – project_completion

The function calculates the percentage of completed tasks for a project. If a project has no tasks, it returns 0.

```
CREATE OR REPLACE FUNCTION project_completion(p_project_id INT)
RETURNS NUMERIC
LANGUAGE plpgsql
AS $$
DECLARE
    v_total INT;
    v_done INT;
BEGIN
    SELECT COUNT(*), COUNT(*) FILTER (WHERE status = 'DONE')
    INTO v_total, v_done
    FROM tasks WHERE project_id = p_project_id;

    IF v_total = 0 THEN
        RETURN 0;
    END IF;

    RETURN ROUND(100.0 * v_done / v_total, 1);
END;
$$;
```

```
SELECT name, project_completion(project_id) AS percent_done
FROM projects ORDER BY project_id;
```

Task Hub Backend	33.3
Task Hub Frontend	0.0
Mobile App	0

9. Stored Procedure – transfer_hours

The procedure transfers budget hours between projects. It validates positive hours, checks that the source project exists, locks the source row using FOR UPDATE, checks available hours, and then performs both updates.

```
CREATE OR REPLACE PROCEDURE transfer_hours(
    p_from INT, p_to INT, p_hours NUMERIC
)
LANGUAGE plpgsql
AS $$
DECLARE
```

```

    v_available NUMERIC;
BEGIN
    IF p_hours <= 0 THEN
        RAISE EXCEPTION 'Hours to transfer must be positive, got %', p_hours;
    END IF;

    SELECT budget_hours INTO v_available
    FROM projects WHERE project_id = p_from FOR UPDATE;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Project % does not exist', p_from;
    END IF;

    IF v_available < p_hours THEN
        RAISE EXCEPTION 'Insufficient budget: project % has % hours, requested %',
            p_from, v_available, p_hours;
    END IF;

    UPDATE projects
    SET budget_hours = budget_hours - p_hours
    WHERE project_id = p_from;

    UPDATE projects
    SET budget_hours = budget_hours + p_hours
    WHERE project_id = p_to;

    RAISE NOTICE 'Transferred % hours from project % to project %',
        p_hours, p_from, p_to;
END;
$$;

```

10. Procedure Execution

```
CALL transfer_hours(1, 3, 15);
-- NOTICE: Transferred 15 hours from project 1 to project 3

CALL transfer_hours(2, 1, 1000);
-- ERROR: Insufficient budget: project 2 has 100.0 hours, requested 1000

SELECT project_id, name, budget_hours
FROM projects ORDER BY project_id;

1 | Task Hub Backend | 85.0
2 | Task Hub Frontend | 100.0
3 | Mobile App | 115.0
```

11. Audit Trigger

An audit table and trigger record INSERT, UPDATE and DELETE operations on tasks. For UPDATE, a record is inserted only when the task status changes.

```
CREATE TABLE task_audit (
    audit_id SERIAL PRIMARY KEY,
    task_id INT,
    operation VARCHAR(6) NOT NULL,
    old_status VARCHAR(12),
    new_status VARCHAR(12),
    changed_by TEXT NOT NULL DEFAULT current_user,
    changed_at TIMESTAMP NOT NULL DEFAULT now()
);

CREATE OR REPLACE FUNCTION log_task_change()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF TG_OP = 'INSERT' THEN
        INSERT INTO task_audit (task_id, operation, new_status)
        VALUES (NEW.task_id, TG_OP, NEW.status);
        RETURN NEW;
    ELSIF TG_OP = 'UPDATE' THEN
        IF NEW.status IS DISTINCT FROM OLD.status THEN
            INSERT INTO task_audit
            (task_id, operation, old_status, new_status)
            VALUES
            (NEW.task_id, TG_OP, OLD.status, NEW.status);
        END IF;
        RETURN NEW;
    ELSE
        INSERT INTO task_audit (task_id, operation, old_status)
        VALUES (OLD.task_id, TG_OP, OLD.status);
        RETURN OLD;
    END IF;
END;
$$;

CREATE TRIGGER trg_task_audit
AFTER INSERT OR UPDATE OR DELETE ON tasks
FOR EACH ROW EXECUTE FUNCTION log_task_change;
```

Audit Output

audit_id	operation	old_status	new_status	changed_by
1	INSERT		TODO	postgres
2	UPDATE	TODO	IN_PROGRESS	postgres
3	UPDATE	IN_PROGRESS	DONE	postgres
4	DELETE	DONE		postgres

12. View – open_tasks_view

```
CREATE OR REPLACE VIEW open_tasks_view AS
SELECT t.task_id, t.title, t.status,
       u.full_name AS assignee, t.due_date
FROM tasks t
LEFT JOIN users u ON u.user_id = t.assignee_id
WHERE t.status <> 'DONE';

SELECT * FROM open_tasks_view ORDER BY due_date;

Build dashboard page | IN_PROGRESS | Ananya Rao
Write FastAPI CRUD | IN_PROGRESS | Arjun Nair
Replace mock data | TODO | Sneha Reddy
Add JWT authentication | IN_PROGRESS | Rahul Verma
Set up CI pipeline | TODO |
Write unit tests | TODO | Rahul Verma
```

13. Materialized View – team_hours_mv

The materialized view stores a snapshot of team-level task hours. Updating a task does not immediately change the materialized view; REFRESH MATERIALIZED VIEW is required.

```
CREATE MATERIALIZED VIEW team_hours_mv AS
SELECT u.team, SUM(t.hours_logged) AS total_hours, COUNT(*) AS tasks
FROM tasks t JOIN users u ON u.user_id = t.assignee_id
GROUP BY u.team;
```

```
SELECT * FROM team_hours_mv ORDER BY team;

UPDATE tasks
SET hours_logged = hours_logged + 10
WHERE title = 'Write FastAPI CRUD';

SELECT * FROM team_hours_mv ORDER BY team; -- old snapshot

REFRESH MATERIALIZED VIEW team_hours_mv;

SELECT * FROM team_hours_mv ORDER BY team; -- refreshed

Before refresh:
Backend | 28.5 | 5
Frontend | 10.5 | 2

After refresh:
Backend | 38.5 | 5
Frontend | 10.5 | 2
```

14. Concurrency Demonstration

Two independent PostgreSQL connections are used to demonstrate concurrency anomalies and isolation levels. The program uses psycopg2 and tests dirty reads, non-repeatable reads, phantom reads, SERIALIZABLE updates, and advisory locks.

Dirty Read – READ UNCOMMITTED requested

```
B updated budget to 999 (uncommitted)
A reads budget = 100.0 -> dirty value NOT visible
PostgreSQL treats READ UNCOMMITTED as READ COMMITTED,
so dirty reads never happen.
```

Non-repeatable Read

```
READ COMMITTED:
A first read = 100.0, B commits 60,
A second read = 60.0 -> ANOMALY (value changed)

REPEATABLE READ:
A first read = 100.0, B commits 60,
A second read = 100.0 -> prevented (same snapshot)
```

Phantom Read

```
READ COMMITTED:
A counts TODO tasks = 2, B inserts one,
A counts again = 3 -> ANOMALY (new row appeared)

REPEATABLE READ:
A counts TODO tasks = 2, B inserts one,
A counts again = 2 -> prevented
```

Serializable Isolation

```
A read 100.0, wrote 90.0 and committed
B read 100.0, tried to write 70.0
-> SerializationFailure: could not serialize access
    due to concurrent update
B must retry the transaction; A's update is not lost.
Final budget = 90.0
```

Advisory Lock

```
A pg_try_advisory_lock(1001) -> True
B pg_try_advisory_lock(1001) -> False (blocked: A holds it)
A unlocks; B tries again -> True
```

Result

The committed transfer was applied atomically, the transaction that violated the CHECK constraint was rolled back completely, and ROLLBACK TO SAVEPOINT undid only the second insert. xmin/xmax showed that an UPDATE creates a new row version (MVCC). Dirty reads never occur in PostgreSQL; non-repeatable and phantom reads occur under READ COMMITTED and are prevented under REPEATABLE READ; SERIALIZABLE raised a serialization failure instead of losing an update. The transfer_hours procedure, project_completion function, audit trigger and materialized view worked as intended.